Ponavljanje

Objasni kratice NLP i NLTK!

Što su tokeni?

Zašto pretvaramo sav tekst u mala slova? Može li se tu pojaviti problem?

Zašto mičemo stop riječi?

Koja je svrha stemizacije i lematizacije?

Što je sentiment analiza?

Uvod

Do sada smo tekst pretvarali u tokene. Ali algoritmi strojnog učenja rade s brojevima. Sljedeći korak je pitanje: kako pretvoriti tekst u numeričku reprezentaciju?

Drugim riječima, model zapravo nije radio nikakvu matematiku nad jezikom. Radili smo samo operacije poput:

- razbijanja teksta na riječi (tokenizacija)
- pretvaranja u mala slova
- uklanjanja stop riječi
- lematizacije
- provjere nalazi li se riječ u nekoj listi

Do sada:

- tekst → tokeni → pravila

Sada prelazimo na:

- tekst → tokeni → vektori → ML model

Prvi pokušaji pretvaranja teksta u brojke

One hot encoding

Korištenje principa iz ML gdje se kategorički podaci pretvaraju u brojčane (spol, razina menadžmenta, kategorija proizvoda).

Problemi:

- ne daje informaciju o odnosima medju podacima
- što je više riječi, raste dimenzionalnost

BoW - Bag of Words

Bag of Words (BoW) je tehnika koja prevodi rečenice, odlomke ili čitave dokumente u fiksne vektore zasnovane na učestalosti riječi.

Ignorira gramatiku i redoslijed riječi, fokusirajući se isključivo na broj riječi u tekstu.

Primjer

He is a good boy.

She is a good girl.

Boy and girl are good.

Imamo tri rečenice u odlomku.

Koraci

Tokenizacija, lowercase, stop words

Ostaju nam riječi koje čine naš vocabulary good boy girl To su naši features.

Radimo Bow matricu. U nazivu reda imamo naše rećenice. U nazivu kolone imamo naše riječi iz vocabularya.

sentence	good	boy	girl
good boy	1	1	0
good girl	1	0	1
boy girl good	1	1	1

Za veće tekstove s više riječi, napravi se vocabular, poredaju se riječi po frekvenciji, od najčešćih, do najrijeđih. Odabere se N najčešćih i samo se njima dodijeli jedinica, sve ostale imaju nulu.

Zadatak

S1: Cats like milk.

S2: Dogs like milk.

S3: Cats like dogs.

Nappraviti vocabulary

Napraviti BOW matricu.

Prednosti tehnike BOW

Jednostavna i intuitivna tehnika za implementaciju

Mane tehnike BOW

Ako dođe do promjene redoslijeda riječi, BOW to ne hvata

BOW ne hvata važnost pojedine riječi

Problemi sa semantikom

Primjer 1

Boy loves dog [1, 1, 1]

Dog loves boy [1, 1, 1]

Primjer 2

S1: The food is good

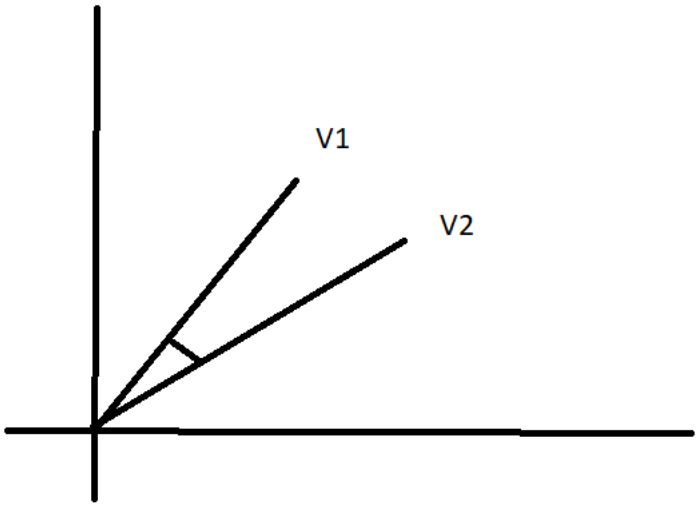
S2: The food is not good

Za ovaj primjer ne mičemo stop riječ "not"

BOW prikaz

V1: [1, 1, 1]

V2: [1, 0, 1]



Pretvorili smo tekst u numeričku reprezentaciju

Dobili smo vektore V1 i V2, koje možemo grafički prikazati

S obzirom na vrlo male razlike u tim vektorima, kad ih grafički prikažemo, oni su vrlo blizu, kut između njih je vrlo mali

S obzirom na njihovu blizinu, mogli bismo reći da su ta dva vektora vrlo slična, da imaju skoro isto značenje

A u stvarnosti, ta dva vektora imaju totalno suprotno značenje

Primjer

za korištenje BOW tehnike koristi se CountVectorizer

```
# --- LIBRARIES ---
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer

# --- MALI CORPUS ---
corpus = [
    "The burger was good and the fries were fresh",
    "The burger was tasty and the service was good",
    "The fries were cold but the burger was still good",
    "The burgers were good and the drinks were fresh",
    "The staff was friendly and the burger was tasty"
]

# --- COUNTVECTORIZER ---
# CountVectorizer ovdje sam radi:
# 1. lowercasing
# 2. tokenizaciju
# 3. uklanjanje stop riječi (jer smo zadali stop_words="english")
# 4. izgradnju vokabulara
# 5. Bag-of-Words matricu

vectorizer = CountVectorizer(stop_words="english")

X = vectorizer.fit_transform(corpus)

# --- VOCABULARY ---
print("Vocabulary:")
print(vectorizer.get_feature_names_out())

print("\nNumber of features (vocabulary size):")
print(len(vectorizer.get_feature_names_out()))

# --- BAG OF WORDS MATRICA ---
```

```
bow_df = pd.DataFrame(
    X.toarray(),
    columns=vectorizer.get_feature_names_out()
)

print("\nBag-of-Words matrix:")
print(bow_df)
```

```
Vocabulary:
['burger' 'burgers' 'cold' 'drinks' 'fresh' 'friendly' 'fries' 'good'
 'service' 'staff' 'tasty']

Number of features (vocabulary size):
11

Bag-of-Words matrix:
   burger  burgers  cold  drinks  fresh  friendly  fries  good  service  \
0       1        0     0       0      1         0      1     1        0
1       1        0     0       0      0         0      0     1        1
2       1        0     1       0      0         0      1     1        0
3       0        1     0       1      1         0      0     1        0
4       1        0     0       0      0         1      0     0        0

   staff  tasty
0      0      0
1      0      1
2      0      0
3      0      0
4      1      1
```

Link na dokumentaciju:

https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html

Imamo riječi burger i burgers, raste nam dimenzionalnost

Koristit ćemo lematizaciju da smanjimo dimenzionalnost

```
# --- LIBRARIES ---
import pandas as pd
import nltk

from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import CountVectorizer

# --- NLTK RESURSI ---
nltk.download("punkt_tab")
nltk.download("wordnet")

lemmatizer = WordNetLemmatizer()

# --- ISTI CORPUS KAO PRIJE ---
corpus = [
    "The burger was good and the fries were fresh",
    "The burger was tasty and the service was good",
    "The fries were cold but the burger was still good",
    "The burgers were good and the drinks were fresh",
    "The staff was friendly and the burger was tasty"
]

# --- FUNKCIJA ZA LEMMATIZACIJU ---
def lemmatize_text(text):

    tokens = word_tokenize(text)

    lemma_tokens = []

    for token in tokens:
        token = token.lower()
        lemma = lemmatizer.lemmatize(token)
        lemma_tokens.append(lemma)

    return " ".join(lemma_tokens)

# --- PRIMJENA NA CORPUS ---
corpus_lemmatized = []

for text in corpus:
    corpus_lemmatized.append(lemmatize_text(text))
```

```
print("Lemmatized corpus:")
print(corpus_lemmatized)

# --- COUNTVECTORIZER ---
# CountVectorizer sada radi:
# tokenizaciju
# uklanjanje stop riječi
# izgradnju vokabulara
# BoW matricu

vectorizer = CountVectorizer(stop_words="english")

X = vectorizer.fit_transform(corpus_lemmatized)

# --- VOCABULARY ---
print("\nVocabulary:")
print(vectorizer.get_feature_names_out())

print("\nNumber of features (vocabulary size):")
print(len(vectorizer.get_feature_names_out()))

# --- BOW MATRICA ---
bow_df = pd.DataFrame(
    X.toarray(),
    columns=vectorizer.get_feature_names_out()
)

print("\nBag-of-Words matrix:")
print(bow_df)
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
Lemmatized corpus:
['the burger wa good and the fry were fresh', 'the burger wa tasty and the service wa good', 'the fry were cold but the burg

Vocabulary:
['burger' 'cold' 'drink' 'fresh' 'friendly' 'fry' 'good' 'service' 'staff'
 'tasty' 'wa']

Number of features (vocabulary size):
11

Bag-of-Words matrix:
   burger  cold  drink  fresh  friendly  fry  good  service  staff  tasty  wa
0       1     0      0      1         0    1     1        0      0      0   1
1       1     0      0      0         0    0     1        1      0      1   2
2       1     1      0      0         0    1     1        0      0      0   1
3       1     0      1      1         0    0     1        0      0      0   0
4       1     0      0      0         1    0     0        0      1      1   2
```

Što radi ovaj kod?

```
# --- LIBRARIES ---
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

# --- MALI DATASET ---
texts = [
    "burger was delicious and fresh",
    "fries were amazing and tasty",
    "burger was cold and terrible",
    "service was slow and bad",
    "burger was very good",
    "fries were awful"
]

labels = [
    "positive",
    "positive",
    "negative",
    "negative",
    "positive",
    "negative"
]

# --- BAG OF WORDS ---
vectorizer = CountVectorizer()

X = vectorizer.fit_transform(texts)
```

```
# --- ML MODEL ---
model = MultinomialNB()

model.fit(X, labels)

# --- TEST REČENICA ---
new_text = ["burger was cold and terrible"]

# pretvaramo tekst u BoW
new_X = vectorizer.transform(new_text)

# predikcija
prediction = model.predict(new_X)

print("Prediction:", prediction[0])
```

```
Prediction: negative
```

Počnite pisati kôd ili generirati uz pomoć umjetne inteligencije.